

DBMS Unit 1 - Introduction to DBMS and Data Modeling

Structured Question Bank

PART A: SHORT ANSWER QUESTIONS (2-3 marks)

Database Management System Basics

Q1: Define Database Management System (DBMS).

- System that stores and manages information about a particular enterprise
- Provides collection of interrelated data along with programs to access it
- Ensures convenient and efficient environment for storing and retrieving data
- Widely used in real life, touching almost every aspect of modern systems

Q2: List any four applications of DBMS.

- **Banking:** Manages transactions securely and consistently
- **Airlines:** Handles flight reservations, schedules, and ticketing
- **Universities:** Supports student registration and academic records
- **E-commerce:** Manages customers, orders, products, and recommendations

Q3: What are the main problems with traditional file systems?

- **Data redundancy and inconsistency:** Same data duplicated with mismatches
- **Difficulty in accessing data:** Every new query requires new program
- **Data isolation:** Information spread across multiple files and formats
- **Integrity problems:** Rules hidden inside program code, difficult to enforce

Database System Architectures

Q4: Define centralized database system.

- Runs on single computer system without interacting with other systems
- Can be single-user (personal computer) or multi-user system
- Uses one to few CPUs connected through common bus
- All database processing occurs on single machine

Q5: What is client-server architecture?

- Divides tasks between client systems (front-end) and server system (back-end)

- Back-end handles access structures, query evaluation, optimization
- Front-end provides tools like forms, report-writers, GUIs
- Communication between front-end and back-end via SQL or APIs

Q6: Differentiate between parallel and distributed systems.

- **Parallel Systems:** Multiple processors and disks connected by high-speed network
- **Distributed Systems:** Data spread across multiple machines connected by network
- Parallel focuses on processing power; distributed focuses on data distribution
- Parallel uses shared resources; distributed uses independent sites

Data Abstraction and Independence

Q7: Define the three levels of data abstraction.

- **Physical Level:** Describes how records are actually stored
- **Logical Level:** Describes data structure and relationships among data
- **View Level:** Provides simplified user perspective hiding implementation details
- Each level hides complexity from levels above it

Q8: What is physical data independence?

- Ability to modify physical schema without changing logical schema
- Applications depend on logical schema, not physical implementation
- Allows storage optimization without affecting applications
- Example: Switching from HDD to SSD without changing queries

Q9: Define schema and instance.

- **Schema:** Logical structure of database (like blueprint)
- **Instance:** Actual data stored at particular point in time
- Schema is static design; instance changes as data is added/deleted
- Example: Student table structure vs actual student records

Database Languages

Q10: What is Data Definition Language (DDL)?

- Used to define and modify database structure (tables, views, indexes)
- Common commands: CREATE, ALTER, DROP, RENAME, TRUNCATE
- Affects only schema (structure), not actual data
- DDL statements are generally not easily reversible

Q11: What is Data Manipulation Language (DML)?

- Used to insert, update, delete, and retrieve data
- Common commands: SELECT, INSERT, UPDATE, DELETE, MERGE
- Two types: Procedural (specify how) and Declarative (specify what)
- DML statements are reversible through rollback operations

Entity-Relationship Model

Q12: What are the basic components of ER model?

- **Entities:** Real-world objects (Student, Employee)
- **Attributes:** Properties of entities (Name, Age)
- **Relationships:** Associations between entities (Student enrolls Course)
- **Symbols:** Rectangles (entities), ovals (attributes), diamonds (relationships)

Q13: Define strong and weak entities.

- **Strong Entity:** Has its own primary key, exists independently
- **Weak Entity:** No primary key, depends on strong entity for identification
- Weak entity uses partial key plus strong entity's primary key
- Example: Employee (strong) and Dependent (weak)

Q14: What are the types of attributes in ER model?

- **Key Attribute:** Uniquely identifies entity (underlined in ER diagram)
- **Composite Attribute:** Can be split into sub-parts (Address → City, State)
- **Multivalued Attribute:** Holds multiple values (Phone Numbers)
- **Derived Attribute:** Calculated from other attributes (Age from DOB)

Q15: Define cardinality constraints.

- **One-to-One (1:1):** One entity relates to exactly one in another set
 - **One-to-Many (1:N):** One entity relates to many in another set
 - **Many-to-One (N:1):** Many entities relate to one in another set
 - **Many-to-Many (M:N):** Many entities relate to many in another set
-

PART B: LONG ANSWER QUESTIONS (5-8 marks)

DBMS Fundamentals and Motivation

Q1: Explain the motivation for Database Management Systems by discussing problems with traditional file systems.

Answer:

1. Data Redundancy and Inconsistency Problems

- Same information stored in multiple files with different formats
- Updates in one file may not reflect in others, causing inconsistencies
- Leads to storage waste and conflicting information
- Example: Customer address stored in multiple department files
- Difficult to maintain synchronized data across different applications

2. Difficulty in Data Access

- Each new query requires writing a new program
- No standard interface for data retrieval
- Time-consuming development process for simple data requests
- Lack of ad-hoc query capabilities
- Programs tightly coupled with data structure, making modifications difficult

3. Data Isolation and Integration Issues

- Information scattered across multiple files in different formats
- Difficult to integrate data from various sources
- Programs written for specific file structures cannot easily access other files
- Lack of centralized data dictionary
- Inconsistent data types and formats across different files

4. Integrity and Business Rule Enforcement

- Business rules (like account balance > 0) embedded in program code
- Difficult to add new constraints or modify existing ones
- No centralized mechanism for constraint enforcement
- Rules scattered across multiple programs, making maintenance complex
- Inconsistent rule application across different applications

5. Concurrency and Security Limitations

- Multiple users accessing same data simultaneously can cause conflicts
- No built-in mechanisms for handling concurrent access

- Difficult to provide user-specific data access permissions
- Limited security features for protecting sensitive information
- Risk of data corruption due to uncontrolled simultaneous access

6. Atomicity and Recovery Issues

- System failures may leave database in inconsistent state with partial updates
- No automatic recovery mechanisms available
- Difficult to ensure all-or-nothing execution of operations
- Manual intervention required for data recovery after system failures
- Risk of permanent data loss due to inadequate backup mechanisms

Database System Architectures Comparison

Q2: Compare and contrast different database system architectures with their advantages and disadvantages.

Answer:

1. Centralized Database Systems

- **Structure:** Single computer system with one or few CPUs and shared memory
- **Advantages:** Simple to manage, lower cost, easier security implementation
- **Disadvantages:** Single point of failure, limited scalability, performance bottleneck
- **Use Cases:** Small organizations, personal applications, single-location businesses
- **Example:** Personal computer databases, small office applications

2. Client-Server Architecture

- **Structure:** Front-end (client) handles user interface, back-end (server) manages database
- **Back-end responsibilities:** Query processing, optimization, concurrency control, recovery
- **Front-end responsibilities:** User interface, forms, reports, application logic
- **Advantages:** Better resource utilization, centralized data management, improved security
- **Disadvantages:** Network dependency, server becomes bottleneck, higher complexity

3. Parallel Database Systems

- **Structure:** Multiple processors and disks connected by high-speed network
- **Types:** Coarse-grain (few powerful processors) vs Massively parallel (thousands of smaller processors)
- **Performance Measures:** Throughput (tasks per unit time) and response time
- **Advantages:** High performance, better resource utilization, scalability
- **Disadvantages:** Complex implementation, higher cost, coordination overhead

4. Distributed Database Systems

- **Structure:** Data spread across multiple sites connected by network
- **Homogeneous:** Same software/schema on all sites, appears as single database
- **Heterogeneous:** Different software/schemas, integrates existing databases
- **Transaction Types:** Local (single site) vs Global (multiple sites)
- **Advantages:** High availability, fault tolerance, local autonomy, scalability

5. Architecture Comparison Summary

Feature	Centralized	Client-Server	Parallel	Distributed
Scalability	Very Limited	Moderate	High	Very High
Fault Tolerance	Low	Medium	High	Very High
Performance	Poor for large users	Better than centralized	Excellent	Depends on network
Complexity	Simple	Moderate	High	Very High
Cost	Low	Medium	High	Variable

Data Abstraction Levels and Data Independence

Q3: Explain the three levels of data abstraction and discuss physical and logical data independence with examples.

Answer:

1. Three Levels of Data Abstraction

- **Physical Level:** Lowest level describing how data is actually stored
 - Deals with record formats, file organization, indexing methods
 - Example: How customer record is stored on disk with specific byte layout
 - Hidden from most database users and application programs
 - Managed by database administrators and system programmers
- **Logical Level:** Middle level describing what data is stored and relationships
 - Defines database schema with entities, attributes, and relationships
 - Example: Instructor(ID, Name, Dept_name, Salary) with specified data types
 - Used by database designers and application developers
 - Independent of physical storage details
- **View Level:** Highest level providing user-specific perspectives
 - Simplifies database by showing only relevant data to specific users
 - Can hide sensitive information for security purposes
 - Different users see different views of same database

- Example: Student view shows courses and grades but not instructor salaries

2. Physical Data Independence

- **Definition:** Ability to change physical schema without affecting logical schema
- **Benefits:** Storage optimization without application changes, flexibility in hardware upgrades
- **Examples of Physical Changes:**
 - Switching storage devices (HDD to SSD or magnetic tapes)
 - Changing file organization techniques or data structures
 - Modifying indexing methods or hashing algorithms
 - Relocating database files to different drives or directories
 - Implementing new compression techniques or storage formats

3. Logical Data Independence

- **Definition:** Ability to change logical schema without affecting external views
- **Challenges:** More difficult to achieve than physical independence
- **Examples of Logical Changes:**
 - Adding, deleting, or modifying attributes in existing tables
 - Creating new entities or relationships in database
 - Merging two records into one or splitting records
 - Changing data types or constraints on attributes
 - Restructuring database schema for normalization

4. Importance of Data Independence

- Provides flexibility for database evolution and optimization
- Reduces maintenance costs by isolating changes to specific levels
- Allows concurrent development of applications and database design
- Enables technology upgrades without affecting existing applications
- Supports long-term database sustainability and growth

5. Practical Implementation

- Database Management Systems provide built-in support for data independence
- Views and stored procedures help achieve logical independence
- File system abstraction enables physical independence
- Standards like SQL help maintain consistency across different implementations
- Proper database design principles enhance both types of independence

Data Models Comparison

Q4: Compare different data models (Hierarchical, Network, Relational, and Object-based) with their characteristics, advantages, and use cases.

Answer:

1. Hierarchical Database Model

- **Structure:** Tree structure with parent-child relationships
- **Characteristics:** Each child has only one parent, efficient for hierarchical queries
- **Advantages:** Simple navigation, efficient for tree-structured data, fast access to hierarchical information
- **Disadvantages:** Rigid structure, difficulty in representing many-to-many relationships, complex data modification
- **Use Cases:** Organization charts, file systems, bill of materials
- **Example:** IBM's IMS (Information Management System)

2. Network Database Model

- **Structure:** Graph structure with nodes (entities) and edges (relationships)
- **Characteristics:** Records can have multiple parents and children, supports complex relationships
- **Advantages:** More flexible than hierarchical, handles many-to-many relationships, efficient navigation
- **Disadvantages:** Complex structure, difficult programming, lacks standardization
- **Use Cases:** Manufacturing systems, supply chain management, complex organizational structures
- **Example:** Students enrolled in multiple courses, courses taught by multiple instructors

3. Relational Database Model

- **Structure:** Data stored in tables (relations) with rows (tuples) and columns (attributes)
- **Characteristics:** Uses SQL for queries, mathematical foundation, supports ACID properties
- **Advantages:** Simple conceptual model, powerful query language, data independence, widespread support
- **Disadvantages:** May not efficiently handle complex data types, object-relational impedance mismatch
- **Use Cases:** Business applications, web applications, transaction processing, data warehousing
- **Examples:** MySQL, Oracle, PostgreSQL, SQL Server

4. Object-Based Database Model

- **Structure:** Data represented as objects with attributes (data) and methods (functions)

- **Characteristics:** Supports inheritance, encapsulation, polymorphism, complex data types
- **Advantages:** Natural representation of complex data, supports multimedia, inheritance reduces redundancy
- **Disadvantages:** Complexity, lack of standardization, performance overhead, limited adoption
- **Use Cases:** CAD/CAM systems, multimedia applications, scientific databases, software engineering
- **Examples:** ObjectDB, db4o, ObjectStore

5. Detailed Comparison Matrix

Feature	Hierarchical	Network	Relational	Object-Based
Structure Flexibility	Rigid	Moderate	High	Very High
Query Complexity	Simple Navigation	Complex Navigation	SQL (Standardized)	OQL (Object Query Language)
Data Integrity	Limited	Better than Hierarchical	Excellent	Excellent with Complex Rules
Performance	Fast for Hierarchical Queries	Good for Complex Queries	Good Overall	Variable (depends on implementation)
Scalability	Limited	Moderate	Excellent	Good
Standardization	Proprietary	Limited	Highly Standardized	Limited Standardization
Development Complexity	Low	High	Moderate	High
Maintenance	Difficult	Very Difficult	Easy	Moderate to Difficult

6. Current Industry Trends

- Relational model dominates commercial database systems
- Object-relational databases combine benefits of both models
- NoSQL databases emerging for specific use cases (document, graph, key-value)
- Hierarchical and network models rarely used in new applications
- Cloud databases primarily based on relational or NoSQL models

Database System Components and Architecture

Q5: Explain the main components of database system architecture including database users, query processing, storage management, and transaction management.

Answer:

1. Database Users Classification

- **Naive Users:** End-users who interact through applications without database knowledge
 - Example: Bank customers using ATMs, online shoppers using e-commerce websites
 - Use pre-defined interfaces and forms for data access
 - Require user-friendly interfaces and error handling
 - Largest group of database users in most organizations
- **Application Programmers:** Develop applications using database APIs and query languages
 - Write code that interfaces with database systems
 - Use programming languages like Java, Python, C# with database connectivity
 - Responsible for application logic and data presentation
 - Need understanding of database design and SQL
- **Sophisticated Users:** Directly interact with database using advanced tools
 - Include data analysts, researchers, business intelligence professionals
 - Use SQL queries, reporting tools, and analytical software
 - Perform complex data analysis and generate custom reports
 - Often create ad-hoc queries for specific business requirements
- **Database Administrators (DBAs):** Manage overall database system operations
 - Responsible for schema design, security, backup, and recovery
 - Monitor database performance and optimize system resources
 - Handle user management, access control, and system maintenance
 - Plan for capacity, upgrade systems, and ensure data integrity

2. Query Processing Components

- **Step 1: Parsing and Translation**
 - Lexical analysis breaks SQL into tokens and identifiers
 - Syntax analysis checks grammatical structure and detects errors
 - Semantic analysis validates table/column names and data types
 - Translation converts SQL into internal algebraic representation
- **Step 2: Query Optimization**
 - Generates multiple equivalent execution strategies
 - Uses cost-based optimization to select most efficient plan
 - Considers available indexes, statistics, and system resources
 - Applies transformation rules to improve query performance
- **Step 3: Query Evaluation**
 - Executes chosen query plan using database engine

- Accesses data using appropriate methods (index scan, table scan)
- Applies join algorithms and processes intermediate results
- Returns final results to requesting application or user

3. Storage Management Responsibilities

- **Interface with File Manager:** Acts as bridge between database and operating system
- **Efficient Data Operations:** Handles storing, retrieving, and updating of data
- **Storage Access Methods:** Implements various access techniques for optimal performance
- **File Organization:** Manages different file structures (heap, sorted, hash, clustered)
- **Indexing and Hashing:** Maintains indexes for faster data retrieval and implements hashing algorithms

4. Transaction Management Components

- **Transaction Definition:** Collection of operations performing single logical function
 - Examples: Bank account transfer, e-commerce order placement, library book issuance
 - Must maintain database consistency and handle concurrent access
 - Ensures all operations complete successfully or none at all
- **Transaction Manager Functions:**
 - Ensures database consistency despite system failures
 - Handles recovery from power failures and system crashes
 - Manages transaction logging and rollback operations
 - Coordinates with concurrency control for multi-user access
- **Concurrency Control Manager Functions:**
 - Controls interaction among concurrent transactions
 - Prevents conflicts when multiple users access same data
 - Implements locking mechanisms and isolation levels
 - Ensures consistency while maximizing system throughput

5. ACID Properties Implementation

- **Atomicity:** Ensures all transaction operations complete or none do
- **Consistency:** Maintains database integrity constraints
- **Isolation:** Prevents interference between concurrent transactions
- **Durability:** Guarantees committed transactions survive system failures
- These properties are enforced through coordination of all system components

Entity-Relationship Model Fundamentals

Q6: Explain the Entity-Relationship model with its components, symbols, and different types of

attributes and relationships.

Answer:

1. ER Model Overview and Purpose

- Conceptual model for designing logical structure of databases
- Represents real-world entities, their attributes, and relationships
- Acts as blueprint for database design before physical implementation
- Provides clear communication tool between designers and stakeholders
- Facilitates systematic approach to database development

2. Basic ER Model Components

- **Entities:** Real-world objects or concepts that can be distinctly identified
 - Examples: Student, Course, Employee, Department, Product
 - Represented by rectangles in ER diagrams
 - Can be strong (independent) or weak (dependent on other entities)
 - Form the foundation of database structure
- **Attributes:** Properties or characteristics that describe entities
 - Examples: Student has Name, ID, Address, Phone
 - Represented by ovals connected to entity rectangles
 - Each attribute has domain defining allowed values
 - Primary key attributes are underlined in diagrams
- **Relationships:** Associations or connections between entities
 - Examples: Student enrolls in Course, Employee works for Department
 - Represented by diamond shapes connecting related entities
 - Can have their own descriptive attributes
 - Define how entities interact in the real world

3. Types of Attributes in Detail

- **Key Attributes:** Uniquely identify each entity instance
 - Underlined in ER diagrams to show primary key
 - Examples: Student_ID, Employee_SSN, Product_Code
 - Must have unique values for each entity occurrence
 - Essential for entity identification and relationship establishment
- **Composite Attributes:** Can be divided into smaller sub-attributes
 - Examples: Name (First_Name, Middle_Name, Last_Name), Address (Street, City, State, ZIP)
 - Represented by ovals with connected sub-attribute ovals

- Provide hierarchical structure to complex attributes
- Can be stored as separate attributes in relational implementation
- **Multivalued Attributes:** Can hold multiple values for single entity
 - Examples: Phone_Numbers, Hobbies, Skills, Email_Addresses
 - Represented by double ovals in ER diagrams
 - Require special handling during conversion to relational schema
 - Often implemented as separate tables in relational databases
- **Derived Attributes:** Calculated from other attributes
 - Examples: Age (from Date_of_Birth), Total_Salary (from Base_Salary + Bonus)
 - Represented by dashed ovals in ER diagrams
 - Can be computed when needed rather than stored
 - Help reduce data redundancy and maintain consistency

4. Relationship Types and Characteristics

- **Binary Relationships:** Most common, involving two entity sets
 - Examples: Student enrolls Course, Employee manages Project
 - Easy to understand and implement in relational databases
 - Support various cardinality constraints (1:1, 1:N, M:N)
- **Ternary Relationships:** Involve three entity sets
 - Examples: Student works on Project under Instructor supervision
 - More complex to implement and understand
 - Cannot always be replaced by multiple binary relationships
- **Recursive Relationships:** Entity relates to itself
 - Examples: Employee supervises Employee, Course has Prerequisites
 - Require special handling and clear role definitions
 - Often involve hierarchical or network structures

5. Cardinality and Participation Constraints

- **Cardinality Constraints:** Define number of entity instances that can participate
 - One-to-One (1:1): Each entity relates to at most one entity in other set
 - One-to-Many (1:N): One entity relates to many entities in other set
 - Many-to-Many (M:N): Many entities relate to many entities in other set
- **Participation Constraints:** Define whether participation is mandatory
 - Total Participation: Every entity must participate (shown with double lines)
 - Partial Participation: Some entities may not participate (single lines)

- Example: Every Course Section must belong to a Course (total), but not every Instructor advises students (partial)

6. Strong vs Weak Entities

- **Strong Entities:** Have their own primary key, exist independently
 - Examples: Student, Employee, Course, Department
 - Represented by single rectangle in ER diagrams
 - Can exist without depending on other entities
- **Weak Entities:** No primary key, depend on strong entities for identification
 - Examples: Dependent (depends on Employee), Course Section (depends on Course)
 - Represented by double rectangles in ER diagrams
 - Use combination of partial key and strong entity's key as composite primary key
 - Connected to strong entity through identifying relationship (double diamond)

Extended ER Features

Q7: Explain the extended ER features: Specialization, Generalization, and Aggregation with examples.

Answer:

1. Specialization Concept and Implementation

- **Definition:** Top-down design process dividing entity set into specialized subgroups
- **Purpose:** Handle entities with distinct characteristics not shared by all members
- **Representation:** Triangle labeled "IS A" connecting superclass to subclasses
- **Attribute Inheritance:** Lower-level entities inherit all attributes and relationships from higher-level
- **Example:** Person entity specialized into Student, Faculty, and Staff subclasses

2. Specialization Characteristics

- **Disjoint vs Overlapping:** Subclasses may be mutually exclusive or allow overlap
- **Total vs Partial:** Every superclass entity must belong to subclass (total) or may not (partial)
- **Additional Attributes:** Subclasses can have attributes not present in superclass
- **Specialized Relationships:** Subclasses can participate in relationships specific to their type
- **Real-world Example:** Employee specialized into Manager, Engineer, and Clerk with specific attributes

3. Generalization Process

- **Definition:** Bottom-up design combining multiple entity sets with common features
- **Purpose:** Reduce redundancy by identifying shared characteristics

- **Relationship to Specialization:** Inverse process but represented identically in ER diagrams
- **Common Attributes:** Shared features moved to higher-level generalized entity
- **Example:** Car, Truck, and Motorcycle generalized into Vehicle superclass

4. Generalization Benefits

- **Reduced Redundancy:** Common attributes stored once in superclass
- **Simplified Queries:** Operations on common attributes performed at superclass level
- **Maintenance Efficiency:** Changes to common attributes made in single location
- **Conceptual Clarity:** Better represents real-world hierarchical relationships
- **Extensibility:** Easy to add new subclasses sharing common characteristics

5. Aggregation Concept and Application

- **Purpose:** Handle situations where relationships need to participate in other relationships
- **Problem Solved:** Eliminates redundancy when relationship sets represent overlapping information
- **Implementation:** Treat relationship as abstract entity that can participate in additional relationships
- **Representation:** Relationship enclosed in rectangle (aggregation box) connected to other entities
- **Higher-order Relationships:** Enables relationships between relationships without duplication

6. Aggregation Example and Benefits

- **Scenario:** Student guided by Instructor on Project, with evaluation of this guidance
- **Without Aggregation:** Separate proj_guide and eval_for relationships with overlapping information
- **With Aggregation:** proj_guide relationship treated as entity connected to evaluation
- **Benefits:**
 - Eliminates redundancy in representing related relationship sets
 - Provides clean abstraction for complex multi-level relationships
 - Maintains clarity while handling higher-order associations
 - Simplifies queries involving multiple relationship levels
 - Reduces complexity in database schema design

7. Extended ER Features Comparison

Feature	Purpose	Direction	Representation	Use Case
Specialization	Handle distinct subgroups	Top-down	IS-A triangle	Person → Student, Faculty
Generalization	Combine similar entities	Bottom-up	IS-A triangle	Car, Truck → Vehicle
Aggregation	Relationships between relationships	Complex modeling	Rectangle around relationship	Project guidance evaluation

ER to Relational Schema Conversion

Q8: Explain the systematic approach to convert ER diagrams into relational schemas with rules and examples.

Answer:

1. Basic Conversion Principles

- Every entity set becomes a relation with same attributes
- Every relationship set becomes relation or is merged with entity relations
- Maintain all constraints and dependencies from ER model
- Ensure resulting schema is in appropriate normal form
- Preserve all information without loss during conversion

2. Step 1: Strong Entity Set Conversion

- Each strong entity directly becomes a relation schema
- Include all simple attributes as columns in the table
- Composite attributes are flattened into separate columns
- Underlined attributes become primary keys
- **Example:** Course entity → course(course_id, title, credits)

3. Step 2: Weak Entity Set Conversion

- Create relation including weak entity's attributes plus foreign key
- Foreign key references the identifying strong entity's primary key
- Composite primary key = strong entity's key + weak entity's partial key
- **Example:** Section weak entity → section(course_id, sec_id, semester, year)
- Primary key: (course_id, sec_id) where course_id is foreign key

4. Step 3: Relationship Set Conversion Rules

- **Many-to-Many (M:N):** Create new relation with primary keys from both entities plus relationship attributes
 - **Example:** advisor(student_id, instructor_id, start_date)

- Primary key: (student_id, instructor_id)
- **One-to-Many (1:N):** Add foreign key from "one" side to "many" side relation
 - **Example:** Instead of separate dept_instructor relation, add dept_name to instructor table
 - More efficient than separate relation for 1:N relationships
- **One-to-One (1:1):** Add foreign key to either side (prefer total participation side)
 - **Example:** manages relationship between Employee and Department
 - Add employee_id to Department table if every department must have manager

5. Step 4: Handling Special Attribute Types

- **Composite Attributes:** Split into component attributes with appropriate naming
 - Name → first_name, middle_initial, last_name
 - Address → street_number, street_name, city, state, zip_code
- **Multivalued Attributes:** Create separate relation for each multivalued attribute
 - **Example:** instructor phone numbers → inst_phone(instructor_id, phone_number)
 - One tuple for each phone number value
 - Primary key: (instructor_id, phone_number)
- **Derived Attributes:** Usually not stored; calculated when needed
 - Age derived from date_of_birth
 - Can be computed in queries or application logic

6. Step 5: Optimization and Redundancy Elimination

- **Total Participation Optimization:** Merge 1:N total relationships using foreign keys
- **Weak Entity Optimization:** Identifying relationship schema usually redundant
- **1:1 Relationship Optimization:** Choose appropriate side for foreign key placement
- **Partial Participation Consideration:** May introduce NULL values in foreign key columns

7. Conversion Example: University Database

Original ER Entities:

- Student(student_id, name, address)
- Course(course_id, title, credits)
- Instructor(instructor_id, name, department)

Relationships:

- Enrolls (M:N between Student and Course with grade)
- Teaches (1:N between Instructor and Course)

Resulting Relations:

- student(student_id, name, address)
- course(course_id, title, credits, instructor_id)
- instructor(instructor_id, name, department)
- enrolls(student_id, course_id, grade)

8. Quality Assurance in Conversion

- Verify all entities and relationships are represented
- Check that all attributes are included in appropriate relations
- Ensure referential integrity constraints are maintained
- Validate that conversion preserves all functional dependencies
- Test schema with sample data to verify correctness
- Review for potential normalization improvements

Summary

This comprehensive question bank covers all major topics in Unit 1 with:

- **15 Short Answer Questions** (2-3 marks each)
- **8 Long Answer Questions** (5-8 marks each)
- Complete coverage of DBMS basics, architectures, data models, ER modeling, and schema conversion
- Each question includes detailed explanations with practical examples
- Structured progression from fundamental concepts to advanced topics
- Real-world applications and implementation considerations
- Clear formatting and logical organization for effective study preparation